

Computer and Network Security

Complete MCQ-Oriented Notes

IBPS SO (IT OFFICER)
COMPLETE PREPARATION SERIES

100% SYLLABUS COVERAGE
CONCEPT-FOCUSED LEARNING
MCQ-ORIENTED APPROACH
QUICK REVISION READY

COMPILED BY
Asabhya Maanav

2026 EDITION
VERSION 1.0

Table of Contents

Computer and Network Security — Complete Notes	5
1. Security Fundamentals	5
1.1 Introduction and Need for Security	5
1.2 Security Goals — The CIA Triad	5
1.3 Security Attacks	6
1.3.1 Passive Attacks	6
1.3.2 Active Attacks	6
1.3.3 Attack Categories by Information Flow	6
1.4 Security Services and Mechanisms (X.800)	6
2. Cryptography Basics	7
2.1 Introduction to Cryptography	7
2.1.1 Attacks on Cryptosystems (Cryptanalytic Attacks)	7
2.2 Classical Encryption Techniques	7
2.2.1 Substitution Ciphers	7
2.2.2 Transposition Ciphers	8
2.2.3 Steganography	8
3. Symmetric Key Cryptography	8
3.1 Symmetric (Secret/Private Key) Concept	8
3.2 Block Cipher vs Stream Cipher	9
3.3 Feistel Cipher Structure	9
3.4 DES — Data Encryption Standard	9
3.5 Double DES and Triple DES (3DES)	9
3.6 AES — Advanced Encryption Standard	9
3.7 Block Cipher Modes of Operation	10
3.8 Stream Cipher — RC4	10
3.9 Key Distribution / Management Problem	10
4. Public Key (Asymmetric) Cryptography	10
4.1 Principles	10
4.2 RSA Algorithm	11
4.3 Diffie-Hellman Key Exchange	11
4.4 ElGamal and ECC	11
4.5 Symmetric vs Asymmetric	11

4.6 PKI and Digital Certificates (X.509)	12
5. Cryptographic Hash Functions	12
5.1 Concept and Properties	12
5.2 Hash Algorithms	12
5.3 Message Authentication Code (MAC) and HMAC	12
5.4 Applications	13
6. Digital Signatures	13
6.1 Concept and Purpose	13
6.2 RSA-based vs DSA	13
6.3 Signature vs Certificate; Non-repudiation	13
7. Authentication	14
7.1 Concept and Factors	14
7.2 Authentication Methods	14
7.3 Authentication Pitfalls and Attacks	14
7.4 Multi-Factor Authentication (MFA)	14
7.5 Kerberos	14
7.6 Mutual Authentication and Challenge-Response	15
8. Network Security Protocols	15
8.1 Overview	15
8.2 SSL / TLS	15
8.3 IPsec	15
8.4 SSH (Secure Shell)	16
8.5 PGP and S/MIME (Email Security)	16
8.6 VPN (Virtual Private Network)	16
9. Network Defense and Threats	16
9.1 Firewalls	16
9.2 IDS and IPS	16
9.3 DMZ (Demilitarized Zone)	17
9.4 Antivirus and Endpoint Security	17
9.5 Malware Types	17
9.6 Network & Application Attacks	17
9.6.1 Man-in-the-Middle (MITM)	17
9.6.2 Replay Attack	17

9.6.3 Denial of Service (DoS) and DDoS	17
9.6.4 Phishing and Social Engineering	18
9.6.5 SQL Injection (SQLi)	18
9.6.6 Cross-Site Scripting (XSS)	18
9.6.7 Cross-Site Request Forgery (CSRF)	18
9.6.8 Spoofing	18
9.6.9 Sniffing	18

10. Quick Revision — Exam Pointers

18

Computer and Network Security — Complete Notes

This guide builds from first principles to advanced exam-grade detail for the **IBPS SO (IT Officer)** examination. Every number, port, key size, round count and core answer is wrapped in red for rapid revision.

1. Security Fundamentals

1.1 Introduction and Need for Security

Computer security is the protection of automated information systems to preserve **integrity**, **availability** and **confidentiality** of information system resources (hardware, software, data, firmware). **Network security** extends this to data **in transit** across networks.

The need for security arises from rising connectivity, valuable digital assets, regulatory pressure, and an expanding **threat landscape** (hackers, insiders, malware, nation states).

Key Terminology

- **Asset** — anything of value (data, hardware, reputation) to be protected.
- **Threat** — a potential cause of an unwanted incident.
- **Vulnerability** — a weakness that a threat can exploit.
- **Risk** — the likelihood that a threat exploits a vulnerability; **Risk = Threat × Vulnerability × Impact**.
- **Attack** — an actual assault that attempts to violate security; **Exploit** — code/technique used to leverage a vulnerability.
- **Countermeasure / Control** — action that reduces risk.

Memory Trick: "TVRAC" — **Threat** finds a **Vulnerability**, creating **Risk**; an **Attack** is realised; a **Countermeasure** reduces it.

1.2 Security Goals — The CIA Triad

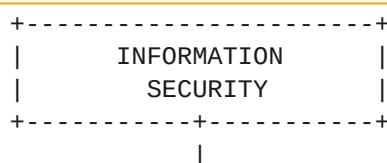
The **CIA triad** is the foundational model of information security with **3** core goals.

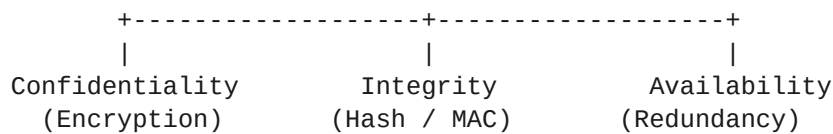
Goal	Meaning	Protects Against	Mechanism
Confidentiality	Only authorised parties read data	Disclosure / eavesdropping	Encryption , access control
Integrity	Data is accurate & unaltered	Modification / tampering	Hashing , MAC, digital signature
Availability	Authorised access when needed	DoS , outages	Redundancy, backups, failover

Two additional widely-cited goals extend the triad:

- **Authentication** — verifying the claimed identity of a user/system.
- **Non-repudiation** — sender cannot deny having sent a message (provided by **digital signatures**).
- **Access Control** — regulating who can access what resource.
- **Accountability / Auditing** — tracing actions to an entity.

MCQ Trap: **Confidentiality** is provided by encryption; **Integrity** by hashing/MAC; **Availability** by redundancy. **Non-repudiation** is NOT part of the core CIA triad — it is an additional service. Examiners swap "integrity" and "confidentiality" definitions frequently.





1.3 Security Attacks

A **security attack** is any action that compromises information security. Attacks are classified as **passive** or **active**.

1.3.1 Passive Attacks

A **passive attack** attempts to learn/use information but does **not** alter system resources. It threatens **confidentiality**.

- **Release of message contents** — reading the actual content of a transmission.
- **Traffic analysis** — observing patterns, frequency, length and parties of messages even when content is encrypted.

Shortcut: Passive attacks are **hard to detect** (no alteration) but **easy to prevent** (encryption). Emphasis is on **prevention**, not detection.

1.3.2 Active Attacks

An **active attack** modifies the data stream or creates a false stream. It threatens **integrity** and **availability**. There are **4** categories:

- **Masquerade** — one entity pretends to be another (impersonation).
- **Replay** — passive capture of data and its subsequent retransmission.
- **Modification of messages** — some portion of a legitimate message is altered, delayed or reordered.
- **Denial of Service (DoS)** — prevents normal use/management of communication facilities.

MCQ Trap: Active attacks are **hard to prevent** but **easy to detect**; emphasis is on **detection + recovery**. Passive = prevention; Active = detection. Do not swap these.

1.3.3 Attack Categories by Information Flow

Attack	Security Goal Violated	Type
Interruption	Availability	Active
Interception	Confidentiality	Passive
Modification	Integrity	Active
Fabrication	Authenticity	Active

Normal:	Source	----->	Destination	
Interruption:	Source	----X	Destination	(cut)
Interception:	Source	----->	Destination	
		--->	Attacker	(eavesdrop)
Modification:	Source	-->	Attacker	--> Destination (alter)
Fabrication:			Attacker	--> Destination (forge)

1.4 Security Services and Mechanisms (X.800)

The **ITU-T X.800** recommendation defines **5** categories of security services:

- **Authentication** — peer entity & data origin authentication.
- **Access Control** — prevents unauthorised use of resources.
- **Data Confidentiality** — protects data from disclosure.

- **Data Integrity** — ensures data is received as sent.
- **Non-repudiation** — proof of origin / proof of delivery.
- (Availability is added by RFC 4949.)

X.800 security mechanisms (specific): **encipherment, digital signature, access control, data integrity, authentication exchange, traffic padding, routing control, notarization.**

Memory Trick: Services answer **WHAT** is protected; Mechanisms answer **HOW** it is protected.
Encipherment = the only mechanism that directly supports confidentiality.

2. Cryptography Basics

2.1 Introduction to Cryptography

Cryptography is the science of writing secret codes; it transforms readable **plaintext** into unreadable **ciphertext** using an algorithm (**cipher**) and a **key**.

Core Terminology

- **Plaintext** — original intelligible message.
- **Ciphertext** — scrambled, unintelligible message.
- **Cipher** — algorithm for encryption/decryption.
- **Key** — secret value controlling the cipher.
- **Encryption (enciphering)** — plaintext → ciphertext.
- **Decryption (deciphering)** — ciphertext → plaintext.

Term	Definition
Cryptography	Designing secure ciphers (making codes)
Cryptanalysis	Breaking ciphers without the key (code-breaking)
Cryptology	The umbrella field = Cryptography + Cryptanalysis

2.1.1 Attacks on Cryptosystems (Cryptanalytic Attacks)

Ordered by increasing attacker advantage:

- **Ciphertext-only** — attacker has only ciphertext (hardest attack).
- **Known-plaintext** — attacker has plaintext–ciphertext pairs.
- **Chosen-plaintext** — attacker can encrypt chosen plaintexts.
- **Chosen-ciphertext** — attacker can decrypt chosen ciphertexts.
- **Brute-force attack** — try every possible key; average $2^{(n-1)}$ attempts for an **n-bit** key.

MCQ Trap: Kerckhoffs's principle — security must depend ONLY on the secrecy of the **key**, never on the secrecy of the algorithm. A cipher is **unconditionally secure** only if ciphertext gives no information (only the **One-Time Pad** qualifies); otherwise it aims to be **computationally secure**.

2.2 Classical Encryption Techniques

2.2.1 Substitution Ciphers

Substitution replaces each plaintext element with another element.

- **Caesar cipher** — shift each letter by a fixed **k** (classically **3**): $C = (P + k) \bmod 26$, $P = (C - k) \bmod 26$. Only **25** usable keys → trivially broken.
- **Monoalphabetic cipher** — arbitrary fixed permutation of the alphabet → $26! \approx 4 \times 10^{26}$ keys, but broken by **frequency analysis**.

- **Playfair cipher** — first practical **digraph** (pair) substitution; uses a **5×5** matrix from a keyword; **I/J share** one cell; 26 letters → 25 cells. Invented by **Charles Wheatstone**, promoted by **Lord Playfair**.
- **Hill cipher** — polygraphic; uses **matrix multiplication mod 26**: $C = KP \bmod 26$; key matrix **K must be invertible mod 26**. Invented by **Lester Hill**.
- **Vigenère cipher** — **polyalphabetic**; uses a repeating keyword and the Vigenère table: $C = (P + K) \bmod 26$. Defeated by the **Kasiski examination** / index of coincidence.
- **One-Time Pad (Vernam)** — key is **truly random**, **as long as the message**, and **used only once** → only cipher with **perfect secrecy** (proved by **Claude Shannon**). Impractical due to key distribution.

Cipher	Type	Key Space / Note
Caesar	Mono-, substitution	25 keys
Monoalphabetic	Substitution	26! keys
Playfair	Digraph substitution	5×5 keyword matrix
Hill	Polygraphic	Invertible matrix mod 26
Vigenère	Polyalphabetic	Keyword length
One-Time Pad	Substitution	Perfect secrecy

Memory Trick: "**Poly = many** alphabets" — **Vigenère** is polyalphabetic; **Caesar/Monoalphabetic** are monoalphabetic. Only **OTP** gives perfect/unconditional security.

2.2.2 Transposition Ciphers

Transposition **rearranges** the positions of plaintext characters without changing them.

- **Rail Fence cipher** — writes text in a zigzag across **N rails**, then reads row by row.
- **Columnar transposition** — writes plaintext in rows under a keyword, then reads columns in the **alphabetical order** of the key letters.

Rail Fence (depth 3) for "WEAREDISCOVERED":

W . . . E . . . C . . . R . .

. E . R . D . S . O . E . E .

. . A . . . I . . . V . . . D

Read rows: WECR ERDSOEE AIVD

2.2.3 Steganography

Steganography hides the **existence** of a message (e.g., LSB image embedding, invisible ink), whereas cryptography hides the **meaning**.

MCQ Trap: **Cryptography** = hide the *meaning* (message visible but unreadable). **Steganography** = hide the *existence* (you don't know a message is there). They can be combined.

3. Symmetric Key Cryptography

3.1 Symmetric (Secret/Private Key) Concept

In **symmetric cryptography** the **same key** is used for encryption and decryption; the key must be shared secretly. It is **fast** and suited to bulk data but suffers from the **key distribution problem**: **n** users need **$n(n-1)/2$** keys.

Plaintext --[Encrypt with SHARED KEY K]--> Ciphertext
Ciphertext --[Decrypt with SAME KEY K]--> Plaintext

3.2 Block Cipher vs Stream Cipher

Feature	Block Cipher	Stream Cipher
Unit	Fixed block (e.g., 64/128 bits)	1 bit / 1 byte at a time
Method	Substitution + permutation	XOR with keystream
Examples	DES, AES, 3DES	RC4, A5/1, ChaCha20
Speed	Slower	Faster
Error propagation	Higher (mode-dependent)	Low

3.3 Feistel Cipher Structure

A **Feistel network** splits each block into **left (L)** and **right (R)** halves and processes them through multiple rounds:

$$L_i = R_{(i-1)}$$

$$R_i = L_{(i-1)} \text{ XOR } F(R_{(i-1)}, K_i)$$

The same structure performs **decryption** by applying subkeys in reverse — its biggest advantage. Design parameters: **block size**, **key size**, **number of rounds**, **subkey generation**, and the **round function F**.

Memory Trick: Feistel = "swap & XOR with F". **DES is Feistel; AES is NOT (it is an SPN).**

3.4 DES — Data Encryption Standard

DES is a **Feistel** block cipher standardised by **NIST (1977)**.

- **Block size = 64 bits; Key size = 64 bits** but only **56 bits effective** (8 are parity bits).
- **16 rounds**; each round uses a **48-bit subkey**.
- Has an **Initial Permutation (IP)** and a final **IP⁻¹**; the round function uses **8 S-boxes** (each 6→4 bits).
- **Insecure today**: brute-forced in 1998; **2⁵⁶** key space is too small.

MCQ Trap: DES key is quoted as **56 bits effective** (64 with parity). Rounds = **16**, block = **64 bits**, subkey = **48 bits**. Do not confuse 56 with 64.

3.5 Double DES and Triple DES (3DES)

- **Double DES** uses two keys (112 bits) but is vulnerable to the **meet-in-the-middle attack**, giving only **~2⁵⁷** effective strength → rarely used.
- **Triple DES (3DES)** applies DES **3 times** in **E-D-E** (Encrypt-Decrypt-Encrypt) mode: **C = E(K3, D(K2, E(K1, P)))**.
- **2-key 3DES** → **112-bit**; **3-key 3DES** → **168-bit** key.
- Block size stays **64 bits**; setting K1=K2=K3 makes it backward-compatible with single DES.

Memory Trick: 3DES = **"EDE"** (Encrypt–Decrypt–Encrypt). Meet-in-the-middle is why we skip plain Double DES.

3.6 AES — Advanced Encryption Standard

AES is the current standard (**NIST 2001**), based on the **Rijndael** cipher by **Daemen & Rijmen**. It is a **Substitution-Permutation Network (SPN)**, **not** Feistel.

- **Block size = 128 bits** (fixed); operates on a **4×4 byte state matrix**.
- **Key sizes = 128 / 192 / 256 bits** → **10 / 12 / 14 rounds** respectively.

- Round operations: **SubBytes** (S-box substitution), **ShiftRows** (cyclic shift), **MixColumns** (GF(2⁸) mixing), **AddRoundKey** (XOR subkey).
- The **final round omits MixColumns**.

AES Variant	Key Size	Rounds
AES-128	128 bits	10
AES-192	192 bits	12
AES-256	256 bits	14

MCQ Trap: AES block size is **always 128 bits** regardless of key size; only the **key size and round count change**. AES is an **SPN, not Feistel**. DES block = 64; AES block = 128.

3.7 Block Cipher Modes of Operation

Mode	Full Name	IV?	Parallel?	Notes
ECB	Electronic Codebook	No	Encrypt+Decrypt	Identical blocks → identical cipher; leaks patterns
CBC	Cipher Block Chaining	Yes	Decrypt only	XOR with previous ciphertext; needs IV
CFB	Cipher Feedback	Yes	Decrypt only	Turns block cipher into self-synchronising stream
OFB	Output Feedback	Yes	No	Stream; no error propagation
CTR	Counter	Nonce	Encrypt+Decrypt	Encrypts a counter; fully parallel, random access

MCQ Trap: **ECB is the least secure** mode (the "ECB penguin" still shows patterns). **CTR and CFB/OFB** turn a block cipher into a stream cipher. CBC requires an **IV**; ECB does not.

3.8 Stream Cipher — RC4

RC4 is a variable-key-length stream cipher by **Ron Rivest (1987)**. It uses a **256-byte state array (S)**, a **Key-Scheduling Algorithm (KSA)** and a **Pseudo-Random Generation Algorithm (PRGA)**. Used in **WEP** and early **SSL/TLS** but now **deprecated** (biased keystream).

3.9 Key Distribution / Management Problem

Symmetric systems must securely share the secret key. Solutions: a trusted **Key Distribution Center (KDC)**, or **asymmetric key exchange** (Diffie-Hellman). The number of keys grows as $n(n-1)/2$, which motivated public-key cryptography.

4. Public Key (Asymmetric) Cryptography

4.1 Principles

Asymmetric cryptography uses a mathematically linked **key pair**: a **public key** (shared openly) and a **private key** (kept secret). What one key encrypts, only the other can decrypt. It solves the **key distribution** problem and enables **digital signatures**. **n** users need only **2n** keys.

Confidentiality: encrypt with RECEIVER's PUBLIC key → decrypt with their PRIVATE key
 Authentication : encrypt/sign with SENDER's PRIVATE key → verify with their PUBLIC key

MCQ Trap: For **secrecy**, encrypt with the **receiver's public** key. For a **signature**, encrypt the hash with the **sender's private** key. Mixing these up is the most common exam error.

4.2 RSA Algorithm

RSA (Rivest, Shamir, Adleman, **1977**) relies on the difficulty of **factoring large integers**.

Key Generation

1. Choose two large primes **p** and **q**.
2. Compute **n = p × q** (the modulus).
3. Compute **φ(n) = (p-1)(q-1)** (Euler's totient).
4. Choose **e** with **1 < e < φ(n)** and **gcd(e, φ(n)) = 1** (public exponent).
5. Compute **d = e⁻¹ mod φ(n)** (private exponent).
6. **Public key = (e, n); Private key = (d, n).**

Encryption / Decryption

- Encrypt: **C = M^e mod n**
- Decrypt: **M = C^d mod n**

Example: p=3, q=11 -> n=33, phi=20
 Choose e=7 (gcd(7,20)=1); d=3 (7*3=21=1 mod 20)
 Encrypt M=2: C = 2⁷ mod 33 = 128 mod 33 = 29
 Decrypt C=29: M = 29³ mod 33 = 2

Memory Trick: "Public encrypts, Private decrypts" — e is public, d is private. **d = e⁻¹ mod φ(n)**.

4.3 Diffie-Hellman Key Exchange

Diffie-Hellman (1976) lets two parties establish a **shared secret** over an insecure channel; its security rests on the **discrete logarithm problem**. It performs **key exchange only** — it does **NOT** encrypt or sign.

Public: prime q, primitive root a (alpha)
 Alice: private x -> public A = a^x mod q
 Bob: private y -> public B = a^y mod q
 Shared key K = B^x mod q = A^y mod q = a^(xy) mod q

MCQ Trap: Diffie-Hellman is **vulnerable to the Man-in-the-Middle attack** because it provides **no authentication**. It does **not** encrypt data — it only exchanges a key.

4.4 ElGamal and ECC

- **ElGamal** — public-key encryption & signatures based on the **discrete logarithm problem**; ciphertext is **twice** the plaintext size; basis of **DSA**.
- **ECC (Elliptic Curve Cryptography)** — based on the **Elliptic Curve Discrete Logarithm Problem (ECDLP)**; gives equivalent security with **much smaller keys**. A **256-bit ECC** key ≈ a **3072-bit RSA** key → ideal for mobile/IoT.

4.5 Symmetric vs Asymmetric

Feature	Symmetric	Asymmetric
Keys	1 shared key	Key pair (public + private)
Speed	Fast (bulk data)	Slow (~1000× slower)
Key count for n users	n(n-1)/2	2n
Key distribution	Hard	Easy (public key shared openly)
Provides	Confidentiality	Confidentiality + authentication + non-repudiation
Examples	DES, AES, RC4	RSA, DH, ECC, ElGamal

Shortcut: Real systems are **hybrid**: asymmetric crypto exchanges a **session key**, then fast symmetric crypto encrypts the bulk data (this is how **TLS** works).

4.6 PKI and Digital Certificates (X.509)

Public Key Infrastructure (PKI) binds public keys to identities using **digital certificates** issued by a **Certificate Authority (CA)**.

- **Digital certificate** — an electronic document binding a public key to an identity, **digitally signed by a CA**.
- **X.509** — the standard certificate format; fields include **version, serial number, signature algorithm, issuer, validity period, subject, subject public key, and CA signature**.
- **PKI components**: **CA** (issues certs), **RA** (Registration Authority — verifies identity), **certificate repository**, and **CRL (Certificate Revocation List) / OCSP** for revocation checking.

MCQ Trap: A **CA signs** certificates; the **RA only verifies** identity (it does not issue). Revocation is checked via **CRL** or **OCSP**. The certificate standard is **X.509**.

5. Cryptographic Hash Functions

5.1 Concept and Properties

A **cryptographic hash function H** maps an arbitrary-length input to a **fixed-length** digest (fingerprint). It is **one-way** and **deterministic**. Required properties:

- **Preimage resistance** — given **h**, infeasible to find **m** with $H(m)=h$ (one-way).
- **Second preimage resistance** — given **m1**, infeasible to find **m2** $\neq$ **m1** with the same hash (weak collision).
- **Collision resistance** — infeasible to find **any** two inputs with the same hash (strong collision).
- **Avalanche effect** — a 1-bit input change flips ~half the output bits.

Memory Trick: Collision resistance is broken first via the **Birthday Attack** — a hash of **n bits** is broken in $\sim 2^{n/2}$ work, not 2^n .

5.2 Hash Algorithms

Algorithm	Digest Size	Block Size	Status
MD5	128 bits	512 bits	Broken (collisions found)
SHA-1	160 bits	512 bits	Broken (SHAttered, 2017)
SHA-256 (SHA-2)	256 bits	512 bits	Secure
SHA-512 (SHA-2)	512 bits	1024 bits	Secure
SHA-3 (Keccak)	224/256/384/512	sponge	Secure (different design)

- **MD5** — by Ron Rivest, **128-bit** digest, **collision-broken**, unsafe for security.
- **SHA-1** — **160-bit**, collision demonstrated (**Google SHAttered 2017**), deprecated.
- **SHA-2** — family (SHA-224/256/384/512); **SHA-256** is the workhorse (used in TLS, Bitcoin).
- **SHA-3** — uses a **sponge construction (Keccak)**, structurally different from SHA-2, standardised 2015.

MCQ Trap: **MD5 = 128 bits; SHA-1 = 160 bits**. Both are broken. SHA-3 is **not** a patch of SHA-2 — it is a different (sponge) design. Hash functions provide **integrity, not confidentiality**.

5.3 Message Authentication Code (MAC) and HMAC

- A **MAC** is a keyed checksum providing **integrity + authentication** (but **not non-repudiation**, since the key is shared).
- **HMAC** is a hash-based MAC defined in **RFC 2104**:

$$\text{HMAC}(K, M) = H((K \text{ XOR } \text{opad}) \parallel H((K \text{ XOR } \text{ipad}) \parallel M))$$

ipad = 0x36 repeated, opad = 0x5C repeated

MCQ Trap: A **MAC** uses a **shared symmetric key** → provides authentication + integrity but **NOT non-repudiation**. A **digital signature** uses a **private key** → it **does** provide non-repudiation.

5.4 Applications

- **Integrity** verification (file checksums, downloads).
- **Password storage** — store **salted hashes** (e.g., bcrypt, PBKDF2, scrypt, Argon2), never plaintext.
- **Digital signatures**, blockchains, and **HMAC** in TLS/IPSec.

6. Digital Signatures

6.1 Concept and Purpose

A **digital signature** is a cryptographic value that provides **authentication, integrity and non-repudiation**. The sender **signs with their private key**; the receiver **verifies with the sender's public key**.

```
SIGN : hash = H(message); signature = Encrypt(private_key, hash)
VERIFY: h1 = Decrypt(public_key, signature); h2 = H(message)
valid <=> h1 == h2
```

MCQ Trap: You **sign with the PRIVATE key** and **verify with the PUBLIC key** — the reverse of encryption for confidentiality. Only the **hash** is signed (not the whole message) for efficiency.

6.2 RSA-based vs DSA

- **RSA signature** — sign by encrypting the message hash with the private key; can also do encryption.
- **DSA (Digital Signature Algorithm)** — part of the **DSS (Digital Signature Standard, FIPS 186)**; based on the **discrete logarithm problem**; **signatures only — cannot encrypt**; faster signing, slower verification than RSA.
- **ECDSA** — the elliptic-curve variant of DSA (small keys).

Feature	RSA	DSA
Basis	Integer factorization	Discrete logarithm
Use	Encrypt + sign	Sign only
Signing speed	Slower	Faster
Verification	Faster	Slower

6.3 Signature vs Certificate; Non-repudiation

- A **digital signature** authenticates a **message**; a **digital certificate** binds a **public key to an identity** (signed by a CA).
- **Non-repudiation** prevents the signer from denying authorship — a unique property of **digital signatures**, not of MACs.

MCQ Trap: Digital **signature** ≠ digital **certificate**. The signature proves who sent *this message*; the certificate proves *who owns a public key*.

7. Authentication

7.1 Concept and Factors

Authentication verifies a claimed identity. The **3** authentication factors are:

- **Something you KNOW** — password, PIN, passphrase (knowledge).
- **Something you HAVE** — token, smart card, OTP device (possession).
- **Something you ARE** — biometrics: fingerprint, iris, face (inherence).
- (Sometimes added: somewhere you ARE — location; something you DO — behaviour.)

Memory Trick: "Know / Have / Are" = the 3 factors. Using two *different* factors = **Two-Factor Authentication (2FA)**; password + PIN is **NOT** 2FA (both are "know").

7.2 Authentication Methods

- **Passwords** — weakest; vulnerable to guessing, dictionary, brute-force.
- **Tokens / Smart cards** — possession-based.
- **Biometrics** — measured by **FAR (False Acceptance Rate)** and **FRR (False Rejection Rate)**; the **CER/EER** is where FAR = FRR.
- **OTP** — one-time passwords; **HOTP** (counter-based, RFC 4226) and **TOTP** (time-based, RFC 6238).

7.3 Authentication Pitfalls and Attacks

- **Password guessing / brute force** — trying many passwords.
- **Dictionary attack** — trying common words/leaked passwords.
- **Rainbow table attack** — precomputed hash→password tables; **defeated by salting**.
- **Phishing** — tricking users into revealing credentials.
- **Credential stuffing** — reusing leaked username/password pairs.
- **Shoulder surfing / keylogging**.

MCQ Trap: A **salt** is a random per-user value added before hashing to defeat **rainbow tables** and ensure identical passwords hash differently. **Pepper** is a secret salt stored separately.

7.4 Multi-Factor Authentication (MFA)

MFA requires **two or more** factors from **different** categories, dramatically reducing account compromise. **2FA** is the common form (e.g., password + OTP).

7.5 Kerberos

Kerberos is a **symmetric-key, ticket-based** network authentication protocol developed at **MIT (Project Athena)**; the current version is **v5 (RFC 4120)**; default port **88**.

- The **KDC (Key Distribution Center)** = **AS (Authentication Server)** + **TGS (Ticket-Granting Server)**.
- A client gets a **TGT (Ticket-Granting Ticket)** from the AS, then requests **service tickets** from the TGS.
- Uses **timestamps** to defeat **replay attacks**; requires **time synchronisation** between hosts.
- Provides **mutual authentication** and **Single Sign-On (SSO)**.
 1. Client -> AS : request (ID)
 2. AS -> Client : TGT + session key (encrypted)
 3. Client -> TGS : TGT + service request

4. TGS -> Client : service ticket
5. Client -> Server : service ticket -> access granted

MCQ Trap: Kerberos uses **symmetric** cryptography (not public-key in the classic design), relies on **timestamps** (so clock skew matters), and runs on **port 88**. KDC = AS + TGS.

7.6 Mutual Authentication and Challenge-Response

- **Mutual authentication** — both parties verify each other (prevents server impersonation).
- **Challenge-response** — verifier sends a random **nonce/challenge**; the prover returns a response computed with a secret, proving knowledge **without sending the secret** (defeats replay).

8. Network Security Protocols

8.1 Overview

Security protocols apply cryptography at different layers: **PGP/S-MIME** (application), **SSL/TLS** (transport), **IPSec** (network).

8.2 SSL / TLS

SSL (Secure Sockets Layer) evolved into **TLS (Transport Layer Security)**; **SSL 3.0 → TLS 1.0**. Current versions are **TLS 1.2** and **TLS 1.3**. TLS secures **HTTP → HTTPS on port 443**.

TLS has sub-protocols:

- **Handshake Protocol** — negotiates cipher suite, authenticates server (and optionally client), and establishes the **session/master secret** (uses asymmetric crypto).
- **Record Protocol** — encrypts bulk application data with the negotiated **symmetric** key.
- **Change Cipher Spec** and **Alert** protocols.

```

Client                                     Server
| --- ClientHello (ciphers) -----> |
| <--- ServerHello + Certificate --- |
| --- key exchange / premaster ----> |
| <==== encrypted session (symmetric) ====> |

```

MCQ Trap: TLS uses **asymmetric crypto for the handshake/key exchange** and **symmetric crypto for bulk data** (hybrid). **HTTPS = HTTP over TLS on port 443**. SSL is deprecated; TLS 1.3 is current.

8.3 IPSec

IPSec secures IP at the **network layer**; it is mandatory in IPv6. Two protocols:

- **AH (Authentication Header)** — **protocol 51**; provides **integrity + authentication** but **NO confidentiality** (no encryption).
- **ESP (Encapsulating Security Payload)** — **protocol 50**; provides **confidentiality + integrity + authentication** (encryption).

Two modes:

- **Transport mode** — encrypts only the **payload**; used host-to-host.
- **Tunnel mode** — encrypts the **entire IP packet** and adds a new header; used for **VPN gateways**.

Key management uses **IKE (Internet Key Exchange)** over **UDP port 500**; associations are called **SAs (Security Associations)** identified by an **SPI**.

Feature	AH	ESP
Protocol number	51	50

Feature	AH	ESP
Confidentiality	No	Yes
Integrity/Auth	Yes	Yes

MCQ Trap: AH = no encryption (authentication only); ESP = encryption + authentication. Tunnel mode is for VPNs (whole packet); Transport mode protects only the payload. IKE uses UDP 500.

8.4 SSH (Secure Shell)

SSH provides encrypted remote login and secure file transfer on port 22, replacing insecure Telnet (23), rlogin and FTP. Current version is SSH-2; it supports password and public-key authentication, port forwarding/tunnelling.

8.5 PGP and S/MIME (Email Security)

- PGP (Pretty Good Privacy) — by Phil Zimmermann (1991); secures email using hybrid crypto: a session key (IDEA/3DES/AES) encrypts the message, RSA/DH encrypts the session key, plus hash (MD5/SHA) and ZIP compression. Uses a decentralised "web of trust" (no central CA).
- S/MIME (Secure/Multipurpose Internet Mail Extensions) — email security using X.509 certificates and a hierarchical CA model (enterprise standard).

MCQ Trap: PGP = web of trust (no CA); S/MIME = hierarchical CA / X.509. Both provide confidentiality + authentication for email and both are hybrid.

8.6 VPN (Virtual Private Network)

A VPN creates a secure encrypted tunnel over a public network, providing privacy and remote access. Tunnelling/security protocols include IPSec, L2TP (often with IPSec), PPTP (port 1723, weak), OpenVPN (SSL/TLS), and WireGuard.

- Site-to-site VPN connects whole networks; Remote-access VPN connects individual users.

9. Network Defense and Threats

9.1 Firewalls

A firewall filters traffic between networks based on a rule set / ACL, enforcing a security policy at the perimeter.

Type	OSI Layer	Inspects	Note
Packet-filtering	L3/L4	IP, port, protocol	Stateless, fast
Stateful inspection	L3/L4	Connection state table	Tracks sessions
Circuit-level gateway	L5	TCP handshake/session	Validates sessions
Application gateway / Proxy	L7	Application content	Deep, slower
NGFW	Multi	App + IPS + identity	Modern unified

MCQ Trap: A packet filter is stateless (judges each packet alone); a stateful firewall remembers connections. An application/proxy gateway works at Layer 7 and inspects content.

9.2 IDS and IPS

- IDS (Intrusion Detection System) — monitors and alerts on suspicious activity; passive (out-of-band).
- IPS (Intrusion Prevention System) — detects and blocks; active/inline.

Detection methods:

- **Signature-based (misuse) detection** — matches known attack patterns; misses **zero-days**.
- **Anomaly-based detection** — flags deviations from a learned baseline; more **false positives**.

Deployment: **NIDS** (network-based) vs **HIDS** (host-based).

MCQ Trap: IDS detects & alerts (passive); IPS blocks (inline/active). Signature-based can't catch zero-days; anomaly-based generates more false positives. A **false positive** = benign flagged as attack; **false negative** = real attack missed.

9.3 DMZ (Demilitarized Zone)

A **DMZ** is a perimeter subnet between the internet and the internal LAN, hosting public-facing servers (web, mail, DNS). It is typically created with **two firewalls** (screened subnet) so a compromised public server cannot directly reach the internal network.

Internet --[Outer FW]-- DMZ (Web/Mail/DNS) --[Inner FW]-- Internal LAN

9.4 Antivirus and Endpoint Security

Antivirus detects malware via **signatures**, **heuristics** and **behaviour monitoring**. **EDR/Endpoint security** adds continuous monitoring and response. Defence-in-depth combines firewalls, IDS/IPS, AV, patching and user training.

9.5 Malware Types

Malware	Self-replicates?	Needs host?	Key Trait
Virus	Yes	Yes (host file)	Needs human action to spread
Worm	Yes	No	Spreads automatically over network
Trojan	No	Yes	Disguised as legitimate software
Ransomware	Varies	No	Encrypts data, demands ransom
Spyware	No	No	Secretly gathers info
Rootkit	No	No	Hides & gains admin/root access

Other: **Adware** (ads), **Bot/Botnet** (remote-controlled army for DDoS), **Logic bomb** (triggers on a condition), **Backdoor** (hidden access), **Keylogger**.

MCQ Trap: A **virus** needs a host file and human help; a **worm** is standalone and self-spreading; a **Trojan** does **NOT** self-replicate. A **rootkit** **hides** to maintain privileged access. **Ransomware** encrypts and extorts.

9.6 Network & Application Attacks

9.6.1 Man-in-the-Middle (MITM)

An attacker secretly relays/alters communication between two parties who believe they talk directly. Countered by **mutual authentication** and **TLS certificates**.

9.6.2 Replay Attack

Captured valid data is retransmitted to gain access. Countered by **nonces**, **timestamps** and **sequence numbers** (as in Kerberos).

9.6.3 Denial of Service (DoS) and DDoS

- **DoS** — a single source floods a target, exhausting bandwidth/CPU/memory → **availability** loss.
- **DDoS** — **distributed** flood from a **botnet** (many compromised hosts).

- Examples: **SYN flood**, **Smurf** (ICMP amplification), **Ping of Death**, **UDP flood**, amplification (DNS/NTP).

9.6.4 Phishing and Social Engineering

Manipulating people into revealing secrets. Variants: **spear phishing** (targeted), **whaling** (executives), **vishing** (voice), **smishing** (SMS), **pretexting**, **baiting**, **tailgating**.

9.6.5 SQL Injection (SQLi)

Injecting malicious **SQL** through unvalidated input to read/modify the database.

Input: ' OR '1'='1

Query: SELECT * FROM users WHERE name='' OR '1'='1'; -> returns ALL rows

Defense: PARAMETERIZED queries / prepared statements + input validation

9.6.6 Cross-Site Scripting (XSS)

Injecting **client-side scripts** into web pages viewed by other users. Types: **Stored** (persistent), **Reflected** (in the request), **DOM-based**. Defence: **output encoding**, input validation, **Content Security Policy (CSP)**.

9.6.7 Cross-Site Request Forgery (CSRF)

Forces an authenticated user's browser to send an unwanted request. Defence: **anti-CSRF tokens**, **SameSite cookies**.

9.6.8 Spoofing

Forging an identity: **IP spoofing**, **ARP spoofing** (LAN MITM via poisoned ARP cache), **DNS spoofing/cache poisoning** (redirects to fake site), **MAC spoofing**, **email spoofing**.

9.6.9 Sniffing

Passive capture of network traffic (e.g., **Wireshark**, **tcpdump**); especially dangerous on **unencrypted** protocols and shared/hub networks.

MCQ Trap: **SQLi** targets the database (server-side); **XSS** targets other users' browsers (client-side script); **CSRF** abuses an existing authenticated session. **DoS** = single source; **DDoS** = botnet (distributed). **ARP spoofing** enables LAN MITM.

10. Quick Revision — Exam Pointers

- CIA = Confidentiality, Integrity, Availability**; +Authentication, +Non-repudiation.
- Passive** attack → prevent (encryption); **Active** attack → detect.
- Symmetric** (DES/AES) = 1 key, fast; **Asymmetric** (RSA/DH/ECC) = key pair, slow.
- DES**: 64-bit block, **56-bit** key, **16** rounds, Feistel. **AES**: 128-bit block, 128/192/256-bit keys, **10/12/14** rounds, SPN.
- RSA** → factoring; **DH/DSA/ElGamal** → discrete log; **ECC** → ECDLP.
- Diffie-Hellman** = key exchange only, vulnerable to **MITM**.
- MD5 = 128-bit**, **SHA-1 = 160-bit**, **SHA-256 = 256-bit** (all integrity).
- Sign with private key, verify with public key**; non-repudiation only from signatures.
- Kerberos**: symmetric, port **88**, KDC = AS + TGS, uses timestamps.
- TLS** port **443** (hybrid); **IPSec**: AH=51 (no encryption), ESP=50 (encryption), IKE UDP 500.
- SSH = 22**, **Telnet = 23 (insecure)**, **PGP = web of trust**, **S/MIME = X.509 CA**.
- Firewall**: packet filter (stateless) vs stateful vs proxy (L7). **IDS** detects; **IPS** blocks.
- Virus** needs host + human; **Worm** self-spreads; **Trojan** disguised; **DDoS** uses a botnet.